

cardiffmet-dat5002-blockchain-fundamentals-solidity

Course:	DAT5002 Blockchain Fundamentals
Authors:	Tom Rowan, ST20285213
	Adam Riley, ST20317308
	Diamond Johnson, ST20316205
Course Ref:	DAT5002_S1_25
Course Tutor:	Dr. Ali Shahaab

Task 3.1

Option (b) Design a generic decentralised blockchain based tender / contracting application.

The application should meet the following requirements (50%): - Allows a set of pre-registered users to setup tender requests for other addresses to bid on. - Allows users to vote (bid) on the campaign. - Only allow one bid per address. - An address can revise their bid as many times as they wish, until the cut-off time. - Assign the tender to the highest bidder once the bidding time is over.

NB - we will assign the tender to the *lowest* bidder.

Project: Bidding System for Pentesting Engagements

Synopsis

Penetration testing is a Cyber Security task that most organisations carry out from time to time.

There are two types of organisations involved: - Customers - Penetration Testing providers ('pentesters')

This application allows customer organisations to raise "Request for Proposals" (RFPs) that describe work that needs to be performed. The application then allows penetration testers to bid for the contract to provide this work.

The register of Customer and Pentesters is maintained by the application owner.

High Level Process

- Customer raises RFP.
- Pentester bid against RFPs during a bidding period.
- After the bidding period is over, the contract is allocated.
- Once the work is completed, and a report delivered, the customer pays the provider.

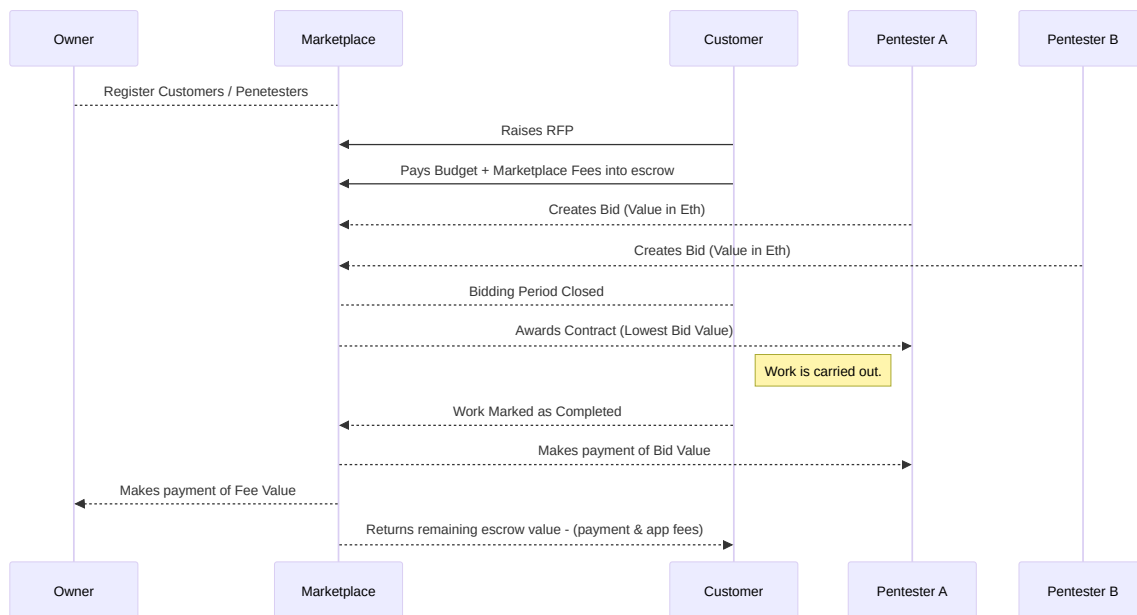
Gas payments

- Gas costs are paid by the caller - a customer when creating a RFP, a pentester when bidding.
- These are tiny amounts when compared with the minimum value of an RFP (1 Eth).
- The web front end would pay the write costs (Owner) out of the received fee, which is 5% of an Eth by default.
- In the real world, we would run a local node to ensure that view and pure functions are gas free.

Web Application

In the real world, we would also run a Web3.js based application which would perform tasks such as: - Taking and validating user input. - Sorting and displaying output from the application to the user. - Handling conversions between user input in Eth, Wei and other fractions of an Eth. - Manage authenticated access to the "RFP Document" and the "Bid/Proposal Document" as alluded to by the URL fields with the data structures. These would be produced by the Customer and bidding Pentester(s) and stored outside of the blockchain by the web application.

Example Application Flow:



A demonstration following this flow is shown [here](#).

Contract and Function Notes

The application is split into a number of contracts to aid readability and to allow the team to work concurrently.

Contract inheritance:



Pentest Marketplace

Purpose: This is the base contract. It defines the main data structures and handles functions to manage the registration of Customers and Pentesters.

Data Structures:

RFP = Request for Proposal

Name	Type	Purpose	Notes
RFP	Struct	Define RFP object	Includes addresses of customer, an expiry date for bidding, the maximum budget available for the project, the URL of supporting documentation and a list of bids.
RFPView	Struct	Define RFPView object	Used exclusively for reporting functions to simplify returning list of RFPs back to the user.
Bid	Struct	Define a 'bid'	Includes the bid value and the URL of the proposal document.
rfps	Mapping	List of RFPs known to the system	
customers	Mapping	List of registered customer addresses	
pentesters	Mapping	List of registered pentester addresses	
customer_rfpCount	Mapping	Count of RFPs by customer	
customer_rfps	Mapping	List of RFPs index number by customer	
pentester_rpfps	Mapping	List of RFPs index	

Name	Type	Purpose	Notes
		number by customer	
rfp_count	uint	Length of the RFP list	Used to index 'next RFP'

Note: it is assumed that there is some external web app that allows for authenticated posting and viewing of RFP scope and response documents. Therefore, the URL fields do not necessarily need to point to 'real documents' on the Internet; no processing is carried out on these URL strings.

Modifiers:

Title	Purpose
onlyValidCustomer	Checks whether sender address is a valid customer
onlyValidPentester	Checks whether sender address is a valid pentester
onlyValidEntity	Checks whether sender address is a valid pentester or customer
hasBudget	Checks whether sender address has an amount in their wallet (Not used)
hasSentEnough	Checks whether right amount has been sent, including fees
onlyValidRFPId	Checks if it's a valid RFP ID

Functions:

Title	Type	Purpose	Can be run by	Requires
addCustomer	External	Add new customer to the register.	Owner	N/A
addPentester	External	Add new pentester to the register.	Owner	N/A
isCustomer	Private	Checks whether an address is a registered customer.	Anyone	N/A
isPentester	Private	Checks whether an address is a registered pentester.	Anyone	N/A
makePayment	Internal	Generic Make Payment Function	Anyone	Contract has funds
calculateFee	Internal	Apply the marketplace fee to a value	Anyone	N/A
changeFee	External	Allow Owner to change the fee	Owner	New fee is between 0 and 100 %
showFee	External	Return current fee value	Anyone	N/A

Title	Type	Purpose	Can be run by	Requires
changeMinimumBudget	External	Allow Owner to change the minimum budget	Owner	New budget is 1 or more Ether
showMinimumBudget	External	Return current minimum RFP budget	Anyone	N/A
viewMarketplaceFunds	External	Allows Owner to see Marketplace value	Owner	N/A
takeMarketplaceFunds	External	Allows Owner to extract funds	Owner	Funds available in Marketplace contract
quickSetup	External	Sets up some customers and pentesters from the remix accounts list.	Owner	Running the contract in Remix environment

Events

Title	Purpose
log_newCustomer	Logs the registration of a new customer.
log_newPentester	Logs the registration of a new pentester.
log_payment	Logs a payment.

Manage RFPs

Purpose:

This contract manages the creation of RFPs by customers.

Data Structures:

No new data structures.

Modifiers:

Title	Purpose
onlyRFPBelongsToCustomer	Check whether the address calling the function is the customer that owns the RFP

Functions:

Title	Type	Purpose	Can be run by	Requires
createRFP	External	Allows customers to create an RFP.	Customers	User sends a "Max Budget" value. User pays (Max Budget + Marketplace Fees) along with the function call.
RFPBelongsToCustomer	Internal	Check whether an RFP id belongs to a specific customer addresse	Anyone	N/A
listCustomerRFPIndexes	External	List the RFPs that belong to a specific customer address	Customers or pentesters	N/A
listCustomerRFPwithTitles	External	List the RFPs that belong to a specific customer address, with titles	Customers or pentesters	N/A

Title	Type	Purpose	Can be run by	Requires
listCustomerRFPsWithDetail	External	List the RFPs that belong to a specific customer address, with more details	Customers or pentesters	N/A

Events

Title	Purpose
log_newRFP	Logs the creation of a new RFP.

Bidding Phase

Purpose:

This contract manages the bidding on RFPs by pentesters.

Data Structures:

No new data structures.

Modifiers:

Title	Purpose
onlyBiddingPeriodOver	Check whether the bidding period is over. Continue if it is.
onlyNotBiddingPeriodOver	// Check whether the bidding period is over. Continue if it is not.

Functions:

Title	Type	Purpose	Can be run by	Requires
makeBid	External	Make a bid. (This also updates a bid)	Pentesters	A valid RFP id, the bidding period is not over, RFP is in Bidding state
getLowestBid	Private	Get the Lowest bidder, and value of their bid	Customer who owns the RFP	A valid RFP id
extendBiddingPeriod	External	Extend the bidding period	Customer who owns the RFP	A valid RFP id, the bidding period is not over
markBiddingCompleted	Private	Mark the bidding process as completed, invoke return payments if no bids	Anyone	A valid RFP id
biddingExpired	Private	Check if the bidding period is over, used in modifier	Anyone	A valid RFP id
			Owner (backend	A valid RFP id, bidding

Title	Type	Purpose	Can be run by	Requires
			server)	period has expired, RFP still in Bidding state
hasbiddingExpired	Private	Check if the bidding period is over	Anyone	A valid RFP id
listAllOpenRFPs	External	List all open RFPs	Pentester	N/A

Events

Title	Purpose
log_newBid	Log a new bid by a pentester
log_bidUpdated	Log a bid being updated by a pentester
log_biddingPeriodExtended	Log customer extending bidding period
log_biddingCompleted	Log setting bidding phase completed flag
log_noBidsReceived	Log that the bidding ended with no bids on the RFP

Pentest Phase

Purpose:

This stage of the contract manages the actions when the pentesting work has been completed for the customer by the pentester. This involves making payments.

Data Structures:

No new data structures.

Modifiers:

Title	Purpose
onlyInWorkPhase	Checks if the RFP is in the Work phase using state

Functions:

Title	Type	Purpose	Can be run by	Requires
pentestCompleted	External	Set the RFP work completed flag to true, will trigger a payment	Customer that owns the RFP	Valid RFP ID, Bidding Phase completed, Testing not already marked completed

Events:

Title	Purpose
log_pentestCompleted	Log setting pentest phase completed flag, shows winning bidder

Reporting

Purpose:

This contract includes reporting and analytics functions for listing out various values. The output would be used to provide values for a web application to utilise in a more user friendly manner.

Data Structures:

Name	Type	Purpose	Notes
CustomerSummary	Struct	View struct for customer summary reporting	
PentesterSummary	Struct	View struct for pentester summary reporting	
BidView	Struct	View struct for bid details	
RFPWithBids	Struct	View struct for RFP with bid details	
PentesterBidHistory	Struct	View struct for pentester's bid history	

Modifiers:

No new modifiers.

Functions:

Title	Type	Purpose	Can be run by	Requires
listAllRFPs	External View	List all detail of all RFPs (minus the bids)	Owner	N/A
listTopPentesters	External View	List all pentesters with their bid activity and earnings	Owner	N/A
getMarketplaceStats	External View	Get overall marketplace statistics	Owner	N/A
viewOwnRFPs	External View	View all RFPs owned by the calling customer with full details	Customers	N/A
viewOwnRFPWithBids	External View	View a specific RFP owned by the customer with all bid details	Customer who owns the RFP	Valid RFP ID, caller must own the RFP
viewBudgetCommitted	External View	View total budget committed by the calling customer across all their RFPs	Customers	N/A

Title	Type	Purpose	Can be run by	Requires
viewCustomerRFPSummary	External View	Get a summary count of RFPs by status for the calling customer	Customers	N/A
viewWinningBids	External View	View all RFPs where the calling pentester won the bid	Pentesters	N/A
viewLosingBids	External View	View all RFPs where the calling pentester bid but did not win	Pentesters	N/A
viewActiveBids	External View	View all RFPs where the calling pentester has an active bid (bidding not complete)	Pentesters	N/A
viewAllMyBids	External View	View all bids placed by the calling pentester (active, won, and lost)	Pentesters	N/A

Title	Type	Purpose	Can be run by	Requires
viewPentesterEarnings	External View	View earnings summary for the calling pentester	Pentesters	N/A
getRFPDetails	External View	Get details of a specific RFP (without bids)	Any registered customer or pentester	Valid RFP ID
getMyBidForRFP	External View	Get the calling pentester's bid for a specific RFP	Pentesters	Valid RFP ID
getTotalRFPCount	External View	Get the total number of RFPs in the system	Any registered customer or pentester	N/A
